

Data Sheet

✓ Height of tripod:
84 cm
+ A4 marker board

✓ Landing pad size:
55cm x 55cm

➔ distance from the
tripod to the edge of the
landing pad:
55 cm

Function Signatur for AR Marker detection

```
corners, ids, _ = cv2.aruco.detectMarkers(frame,  
                                          ARUCO_DICTIONARY)
```

Parameters

- ✓ frame (np.ndarray): An 8-bit single-channel (grayscale) or 3-channel (BGR color) image (uint8) in which ArUco markers will be detected.
- ✓ ARUCO_DICTIONARY (cv2.aruco.Dictionary): The predefined or custom ArUco dictionary object defining the target marker set to search for (e.g., cv2.aruco.getPredefinedDictionary(...)).

Return Value (corners, ids)

- ✓ corners (list of np.ndarray): Type: list of np.ndarray (or an empty list if no markers are detected).
 - ✓ Data Structure: Each item in the list represents a detected marker's 4 corners with shape (1, 4, 2).
 - ✓ Coordinates Format: Floating-point pixel positions formatted as [[[x1, y1], [x2, y2], [x3, y3], [x4, y4]]], ordered clockwise starting from the top-left corner.
- ✓ ids (np.ndarray or None):
 - ✓ Type: 2D np.ndarray with shape (number_of_detected_markers, 1) and type int32 (or None if no markers are detected).
 - ✓ Data Structure: Contains the unique integer identifier corresponding to each detected marker in corners.

Function Signatur for HSV Color detection

```
contours =  
cv2.findContours(pink_mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)[-2]
```

Parameters

- ✓ pink_mask (np.ndarray): An 8-bit single-channel binary image (uint8). Non-zero pixels are treated as foreground objects, while zero pixels are background.
- ✓ cv2.RETR_EXTERNAL (int): Retrieval mode that extracts only the extreme outer contours, ignoring all nested child contours
- (holes).cv2.CHAIN_APPROX_SIMPLE (int):
 - ✓ Contour approximation method that compresses horizontal, vertical, and diagonal segments, keeping only their end/corner points to save memory.

Return Value (contours)

- ✓ Type: list of np.ndarray (or an empty tuple / list if no targets are found).
- ✓ Data Structure: Each item in the list represents a single detected object boundary with shape (number_of_vertex_points, 1, 2).
- ✓ Coordinates Format: Integer pixel positions formatted as [[[x1, y1]], [[x2, y2]], ...].
- ✓ Data Behavior: Due to cv2.CHAIN_APPROX_SIMPLE, only the key corner/boundary points are stored (e.g., a simple rectangle contains exactly 4 points).

Introduce for drone's APIs for Hand Sign Control Task

[API-docs\(PDF\)](#)

Supplementary Information on AR Marker Detection



```
corners, ids, rejectedImgPoints = cv2.aruco.detectMarkers(img, dictionary)
```

Variables Used Below

markerNumber: Number assigned to each marker in the order detected (0-indexed).

cornerNumber: Corner number starting from the top-left in normal orientation, going clockwise (0 to 3).

```
ids[markerNumber][0]
```

➔ ID of the **markerNumber**-th detected marker

```
corners[markerNumber][0][cornerNumber]
```

➔ the **markerNumber**-th detected marker:

➔ the **cornerNumber**-th corner:

➔ [x, y] coordinates

```
corners[markerNumber][0][cornerNumber][0] ➔ x-coordinate
```

```
corners[markerNumber][0][cornerNumber][1] ➔ y-coordinate
```

Method for the measurement of the detection parameters:

1. Comment out the "single_fly_takeoff(self, led=0)"
2. Add the print(f"{parameter1},{parameter2},...") to the next line of the detection function
3. Run the program and carry the drone with your hand